

Falcon Tech Service

Техническое руководство по поисковому сервису

Назначение: принять фотографию и bbox автомобиля, найти его изображения в подготовленной галерее и явно ответить «нет в галерее», когда совпадение не подтверждено.

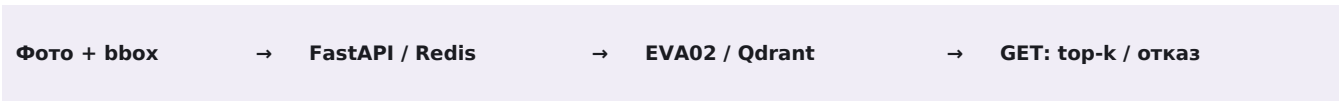
Приём	Поиск	Ответ
202 + task_id Асинхронный HTTP-контракт	Redis → EVA02 → Qdrant Воркер вне API-процесса	top-k или отказ GET результата по task_id

React-интерфейс помогает выбрать исходный кадр и рамку автомобиля. FastAPI валидирует запрос и ставит задачу в Redis Streams. Независимые inference-воркеры загружают модель, извлекают 256-мерный признак, выполняют поиск по Qdrant и решают, подтверждено ли совпадение. Исходные изображения лежат в S3-совместимом RustFS, история и метаданные - в Postgres.

Этот документ относится к **демонстрационному сервису** из LCT-Falcon-Tech. Конкурсный offline Docker, который создаёт submission.csv, candidates.csv и embeddings.npy, находится в сабмодуле training/ и описан отдельно в ML-руководстве. Сервис не создаёт эти три файла.

Версия документа: 28.09.2026. Источник: LCT-Falcon-Tech, commit 236a821; данные и команды сверены с README.md, docker-compose.yaml, .env.example и реализацией сервисов.

Путь одного запроса



Галерея индексируется до старта API: init загружает веса и изображения, воркеры считают признаки, gallery-ready дожидается завершения.

Содержание

01. Что поставляется и где проходит граница	3
Четыре стадии запроса	3
02. Запуск по шагам	4
2.1. Требования и подготовка	4
2.2. Подключить галерею	4
2.3. Поднять стек и убедиться в готовности	5
03. Архитектура и границы компонентов	6
Структура репозитория	6
04. Жизненный цикл поискового запроса	7
4.1. POST: ingestion без ожидания модели	7
4.2. Worker: processing, analysis, result	7
4.3. GET и устойчивость очереди	7
05. Как строится и обновляется галерея	8
5.1. Одноразовый init	8
5.2. Готовность, сбои и принудительный пересчёт	8
06. Модель, поиск и open-set отказ	9
6.1. Что именно решает refusal	9
07. HTTP API и контракт данных	10
7.1. Маршруты и статусы	10
7.2. Поля готового результата	10
08. Frontend: от рамки к результату	11
Polling и состояния интерфейса	11
09. Хранилища и срок жизни состояния	13
10. Конфигурация и развёртывание	14
Инфраструктурные образы и доступ	14
11. Масштабирование и эксплуатационные границы	15
Наблюдаемость	15
12. Проверки и воспроизводимость	16
12.1. Минимальная сквозная проверка	16
12.2. Тесты и сборка разработчика	16
13. Диагностика типовых ошибок	17
14. Карта исходников	18

01. Что поставляется и где проходит граница

В репозитории находятся два связанных, но разных сценария. **Демонстрационный сервис** отвечает на одиночные пользовательские запросы через HTTP и веб-интерфейс. **Конкурсный submission** запускается отдельным offline образом из training/: он пакетно обрабатывает test query/gallery и сохраняет три файла. Оба используют ту же обученную EVA02 и пресет отказа, но разные интерфейсы и режимы запуска.

Сценарий	Вход	Выход	Где
Сервис	Изображение + bbox + top_k	task_id; затем JSON с кандидатами либо rejected	frontend/, backend/, inference/
Конкурс	CSV query/gallery + изображения	submission.csv, candidates.csv, embeddings.npy	training/

Сервис не обучает модель на лету. Он загружает экспортированный serving checkpoint eva02.pt, а init проверяет его SHA256. Источник метрик качества и протокол валидации - ML-репозиторий, не HTTP-демо.

Четыре стадии запроса

Стадия	Кто выполняет	Действие
Ingestion	backend	Проверка файла и bbox, сохранение оригинала в RustFS, публикация задачи.
Processing	inference	Кроп, нормализация, EVA02 embedding размерности 256.
Analysis	inference	Cosine-поиск в Qdrant, решение open-set отказа.
Result	inference → backend	Сортировка кандидатов, запись результата в Redis, чтение через GET.

Номерные знаки в исходных данных заблюрены и не используются как признак. Поиск опирается на внешний вид автомобиля. Поле vehicle_id, если оно есть в манифесте, служит метаданными ответа, а не фильтром поиска.

02. Запуск по шагам

2.1. Требования и подготовка

- Docker Engine с Compose v2; для GPU - NVIDIA Container Toolkit. CPU-режим предусмотрен, но индексация большой галереи может быть долгой.
- Достаточно места для весов (eva02.pt около 1.2 ГБ), данных Postgres, Redis, Qdrant и RustFS. Каждая inference-реплика держит собственную копию модели в памяти.
- Сеть на первом старте нужна init для скачивания весов с Hugging Face, если файлов нет в data/weights.

```
git clone --recurse-submodules https://github.com/gruzdev-as/LCT-Falcon-Tech.git
cd LCT-Falcon-Tech
# если репозиторий уже скопирован:
git submodule update --init
cp .env.example .env
```

Значения .env.example рассчитаны на localhost. Для локального старта достаточно оставить их без изменений, но галерею надо подключить отдельно. Для внешнего хоста поменяйте секреты, публичный S3 endpoint и сетевую экспозицию портов.

2.2. Подключить галерею

```
data/gallery/
manifest.csv
images/
  car_001.jpg
  camera_03/car_002.png
```

GALLERY_DIR по умолчанию равен ./data/gallery; Compose монтирует этот каталог в init как /gallery:ro. Можно указать другой абсолютный или относительный путь в .env. Файлы в images/ - исходные кадры, не готовые кропы. Модель применяет bbox одинаково к галерее и запросу.

```
image_id,image_path,vehicle_id,camera_id,bbox_x,bbox_y,bbox_width,bbox_height
car_001,car_001.jpg,,120,265,588,482
car_002,camera_03/car_002.png,42,cam_03,10,20,300,180
```

Поле	Правило
image_id	Уникальная строка; возвращается клиенту. Дубли не принимаются.
image_path	Путь относительно images/; указанный файл должен существовать.
vehicle_id, camera_id	Колонки обязательны, значения могут быть пустыми.
bbox_x/y/width/height	Абсолютные пиксели хуwh; ширина и высота не меньше 16.

CSV: UTF-8, допустим BOM, разделитель - запятая, первая строка - заголовки. Все восемь колонок обязательны; порядок не важен. Дополнительные колонки игнорируются. init валидирует схему, уникальность ID и наличие изображений до постановки задач.

2.3. Поднять стек и убедиться в готовности

```
docker compose up --build -d
docker compose logs -f gallery-ready
# GPU-вариант:
docker compose -f docker-compose.yaml -f docker-compose.gpu.yaml up --build -d
# масштабирование воркеров:
INFERENCE_REPLICAS=4 docker compose up -d
```

Порядок Compose: инфраструктура → init → inference → gallery-ready → backend → frontend.

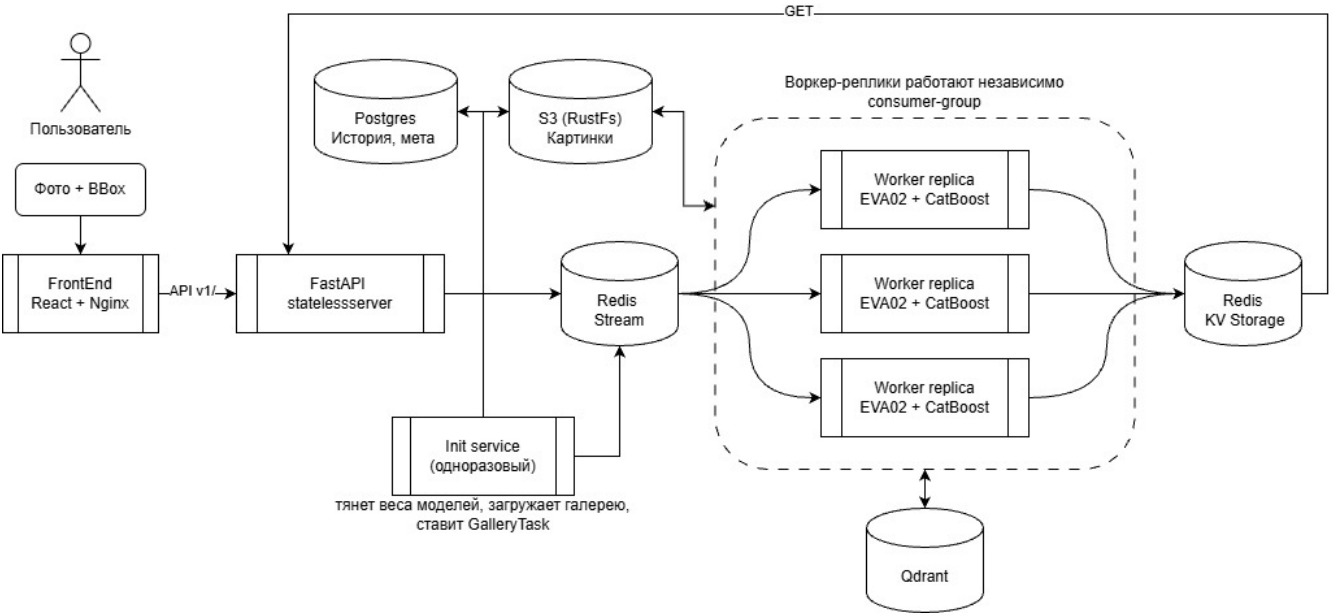
gallery-ready завершается только когда ожидаемые изображения проиндексированы или помечены как битые; нулевая пригодная галерея считается ошибкой. На CPU индексация идёт по одному кадру на задачу и может занять значительное время.

Проверка	Адрес / команда
Веб-интерфейс	http://localhost:8080
OpenAPI / Swagger	http://localhost:8000/docs
Backend liveness	<code>curl http://localhost:8000/health</code>
Прогресс галереи	<code>docker compose logs -f gallery-ready</code>
Состояние контейнеров	<code>docker compose ps</code>

/health проверяет живость FastAPI, но **не** доступность Redis, Qdrant, Postgres или worker. Для полноценной проверки дождитесь gallery-ready, выполните тестовый POST и получите завершённый GET.

03. Архитектура и границы компонентов

Схема ниже взята из docs/images/architecture.jpg. Слева пользовательский ввод, в центре stateless FastAPI и хранилища, справа независимые воркеры одной consumer group. Верхний контур GET показывает чтение готового результата из Redis KV; нижний init-контур - подготовку галереи.



Исходная схема сервиса: FrontEnd, FastAPI, Postgres, RustFS, Redis Streams/KV, worker replicas, Qdrant и одноразовый init. Источник: docs/images/architecture.jpg.

Компонент	Ответственность
Frontend: React + nginx	Выбор кадра и bbox, submit/polling, визуализация результатов и отказа; nginx проксирует /api/.
Backend: FastAPI	HTTP-контракт, валидация и S3 upload, публикация в Redis, чтение результата, история в Postgres. Модель не загружает.
Inference worker xN	Держит EVA02 и CatBoost; читает два Redis Stream, индексирует gallery и обслуживает поиск.
Init + gallery-ready	Скачивает артефакты, синхронизирует галерею, ставит задания; затем ждёт готовности индекса.
Redis / Qdrant / Postgres / RustFS	Очередь и короткоживущие статусы / векторы / метаданные / оригинальные изображения.

Структура репозитория

backend/	app/	src/	deploy/	tests/
inference/	app/	src/	deploy/	tests/
init/	app/	src/	deploy/	tests/
common/	src/	tests/		
frontend/	src/	deploy/	tests/	
training/	Git submodule: модель и конкурсный offline Docker			

04. Жизненный цикл поискового запроса

4.1. POST: ingestion без ожидания модели

POST /api/v1/search принимает multipart file, JSON-строку bbox и top_k. Backend проверяет реальный формат изображения, размер файла, пиксельные размеры и геометрию bbox. Затем сохраняет исходный файл в RustFS под queries/, ставит ключ task:{id}=pending с TTL 1 час, публикует EmbeddingTask в falcon:tasks и best-effort записывает метаданные в Postgres. Ответ - HTTP 202 и task_id.

```
POST /api/v1/search
file = исходное изображение
bbox = {"x":10,"y":10,"width":200,"height":150}
top_k = 10
→ 202 {"task_id":"...", "status":"pending"}
```

4.2. Worker: processing, analysis, result

Worker ставит статус processing, читает оригинал из RustFS, вырезает автомобиль, применяет те же transforms и конфигурацию ТТА, которые сохранены в serving checkpoint, L2-нормирует 256-D признак. Qdrant возвращает не меньше max(top_k, neighbours_for_refusal) ближайших точек; CatBoost-режим при необходимости получает также их векторы. Воркер применяет отказ, сортирует кандидатов и сохраняет SearchResult в Redis на час. Только после записи результата он подтверждает сообщение Stream через ХАСК.

Результат	Смысл
status=done, rejected=false	Кандидаты отсортированы по убыванию score; слабые ниже min_score удалены.
status=done, rejected=true	Совпадение не подтверждено; candidates=[]. Это штатный open-set ответ.
status=failed	Ошибка обработки конкретного запроса; смотреть error, а не трактовать пустую выдачу как отказ.

4.3. GET и устойчивость очереди

GET /api/v1/search/{task_id} сначала читает result:{id}. При готовности он best-effort сохраняет итог и кандидатов в Postgres один раз и отвечает 200. Пока есть task:{id}, отвечает 202 с processing. Если оба ключа отсутствуют, отвечает 404: ID неизвестен или TTL истёк.

Все inference-реплики входят в consumer group inference-workers: одну задачу берёт одна реплика. Зависшее сообщение может быть перехвачено после INFERENCE_CLAIM_IDLE_MS; после нескольких доставок «ядовитая» задача получает failed. Приоритет у поисковых задач; галерея индексируется, когда нет поиска. Новый поиск, пришедший во время индексации одного кадра, ждёт завершения этого кадра.

latency_ms в результате измеряется от публикации задачи до её завершения и включает ожидание в очереди, загрузку, forward и постобработку. Это не чистое время модели и не конкурсный latency из ML-руководства.

05. Как строится и обновляется галерея

5.1. Одноразовый init

Init проверяет наличие `eva02.pt` и CatBoost-файла: по URL из `.env.example` скачивает их с Hugging Face, сверяет SHA256, создаёт схему Postgres, читает манифест, заливает оригиналы в S3 и синхронизирует таблицу `gallery_images`. Затем сверяет Qdrant с желаемым набором изображений и версией модели, удаляет устаревшие точки и публикует только отсутствующие `GalleryTask` в `falcon:gallery`. Сам `init` эмбединги не считает.

```
init → veca + checksum → manifest → RustFS + Postgres  
      → свёрка Qdrant → falcon:gallery  
inference workers → crop + EVA02 → upsert Qdrant  
gallery-ready → count(model_version) + failed == total → backend
```

`model_version` - SHA256 serving-весов; каждая Qdrant-точка хранит эту версию. При смене весов существующий индекс не смешивается с новым: точки старой версии удаляются и пересчитываются. Хеш манифеста `gallery_version` фиксирует состояние метаданных. Повторный старт с теми же данными пропускает уже готовые объекты и точки.

5.2. Готовность, сбой и принудительный пересчёт

`gallery-ready` сравнивает ожидаемое число изображений с количеством Qdrant-точек нужной версии и множеством `gallery:failed`. Битые изображения исключаются из поиска и перечисляются в предупреждении. Если не проиндексировано ни одного кадра, ожидание завершается ошибкой. При остановке прогресса появляется `warning`, но `INIT_WAIT_TIMEOUT_S=0` означает ждать без общего лимита.

```
docker compose logs -f init inference gallery-ready  
INIT_FORCE=true docker compose up -d  
# эквивалент для одноразового bootstrap:  
docker compose run --rm init --force
```

`INIT_FORCE=true` удаляет коллекцию Qdrant и ожидающие `gallery`-задачи перед пересчётом. Это дорогая операция; штатный повторный `up` идемпотентен и в большинстве случаев не требует `force`.

06. Модель, поиск и open-set отказ

Рабочий `INFERENCE_EMBEDDER=eva02` загружает экспорт `reid-serving` из `training/` и сохранённую в нём конфигурацию. Инференс использует PIL decode, тот же `bbox crop`, трансформации и fusion ТТА-вью из ML-пакета. Устройство `auto` выбирает CUDA при доступности; на CPU используется `fp32`, на CUDA `bf16`. `eval.fast_kernels=false` фиксируется в адаптере. `torch.compile` включается отдельно через `INFERENCE_COMPILE=true` только на CUDA.

Query и gallery проходят общий `embed_vector`: выход приводится к `float32`, проверяется размерность и L2-нормируется. Коллекция Qdrant использует `cosine`. Точка галереи хранит `image_id`, `image_path`, `bbox`, необязательные `vehicle_id` и `camera_id`, а также `model_version`.

6.1. Что именно решает refusal

Пресет	Условие принятия
<code>eva02_ensemble.yaml</code> (дефолт)	Лучший <code>cosine</code> $\geq$ порог и <code>CatBoost P(match)</code> $\geq$ порог модели.
<code>eva02_model.yaml</code>	Решение CatBoost-головы без <code>cosine AND</code> .
<code>eva02_threshold.yaml</code>	Один <code>cosine</code> -порог.
<code>INFERENCE_REJECT_THRESHOLD</code>	Ручной <code>override</code> : один заданный <code>cosine</code> -порог вместо пресета.

CatBoost отвечает на вопрос «есть ли такой автомобиль в галерее» по `query` и ближайшим `gallery`-соседям; это отдельная модель, а не часть EVA02 forward. Refusal сначала решает, принимать ли запрос в целом, затем `min_score` отсеивает слабых кандидатов. Если запрос отклонён, API возвращает пустой список, сохраняя `top_score` для диагностики.

Отображаемый интерфейсом процент - `score` поиска, приведённый к интервалу `[0,1]` и умноженный на 100. Не следует читать его как калиброванную вероятность идентичности. `CatBoost P(match)` - отдельный сигнал для отказа.

07. HTTP API и контракт данных

7.1. Маршруты и статусы

Маршрут	Ответ
POST /api/v1/search	202: task_id, status=pending; при неверном вводе 422.
GET /api/v1/search/{task_id}	202: processing; 200: SearchResult; 404: неизвестный или истёкший ID.
GET /health	200: живость FastAPI без обращения к зависимостям.
GET /docs, /redoc	Интерактивное описание FastAPI.

POST принимает JPEG, PNG, WebP или BMP размером до 20 МиБ, не более 50 млн пикселей и с каждой стороной не меньше 16 px. Bbox передаётся как JSON-объект `x,y,width,height` либо массив `[x,y,width,height]`; координаты в исходных пикселях от левого верхнего угла. Бокс обрезается по рамке изображения, после обрезки каждая сторона должна оставаться ≥ 16 px. `top_k` - целое от 1 до 100, по умолчанию 10.

```
curl -F file=@car.jpg \
  -F 'bbox={"x":10,"y":10,"width":200,"height":150}' \
  -F top_k=10 http://localhost:8000/api/v1/search
# взять task_id из ответа, затем опрашивать:
curl -i http://localhost:8000/api/v1/search/<task_id>
```

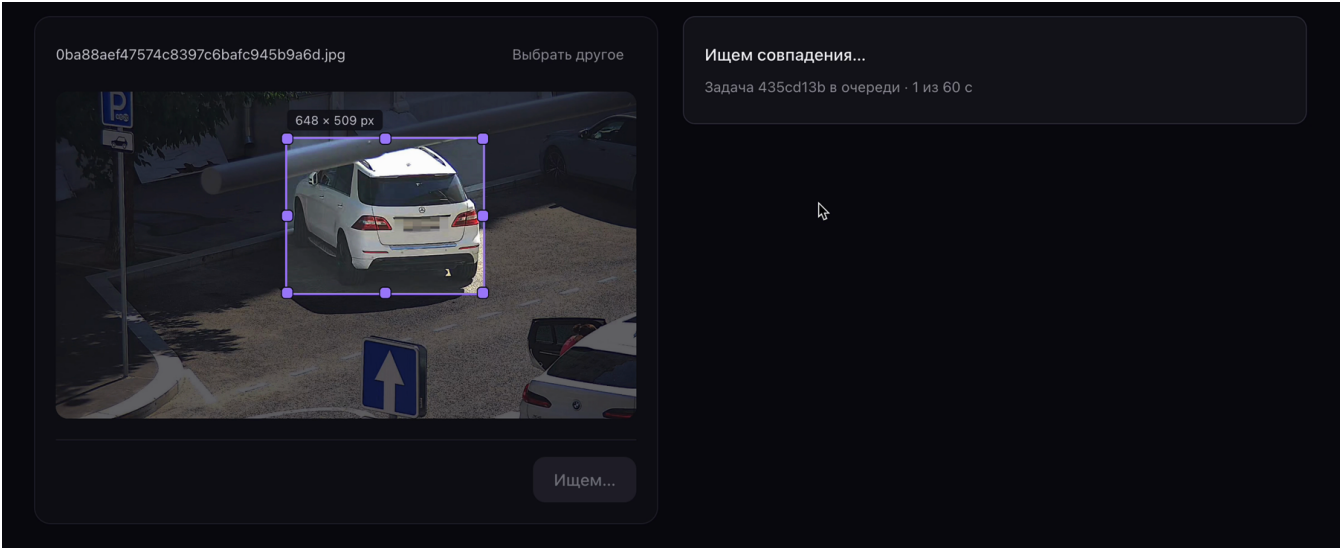
7.2. Поля готового результата

Поле	Значение
status, rejected, error	done или failed; отдельный флаг штатного отказа; причина ошибки только при failed.
top_score	Similarity лучшей Qdrant-точки либо null при пустой галерее.
candidates[]	image_id, score, rank, URL исходного кадра, bbox и метаданные.
model_name, latency_ms	Идентификатор эмбеддера и длительность от постановки задачи.

`image_url` - presigned ссылка на RustFS для браузера. По умолчанию действует 1 час (`S3_PRESIGN_TTL`). После истечения ссылки новый GET результата может вернуть ранее сохранённый URL; для долгосрочного экспорта следует отдельно переподписывать ссылки. Ошибки домена возвращают `JSON error, message, details`.

08. Frontend: от рамки к результату

React + TypeScript приложение собирается Vite и выдаётся nginx на порту 8080. Пользователь загружает кадр, рисует или меняет bbox, запускает поиск. Координаты хранятся в пикселях оригинала, а не CSS-изображения: при масштабировании окна рамка пересчитывается из исходных координат. Клиент дублирует быстрые проверки файла и bbox для обратной связи, но окончательное решение принимает backend.

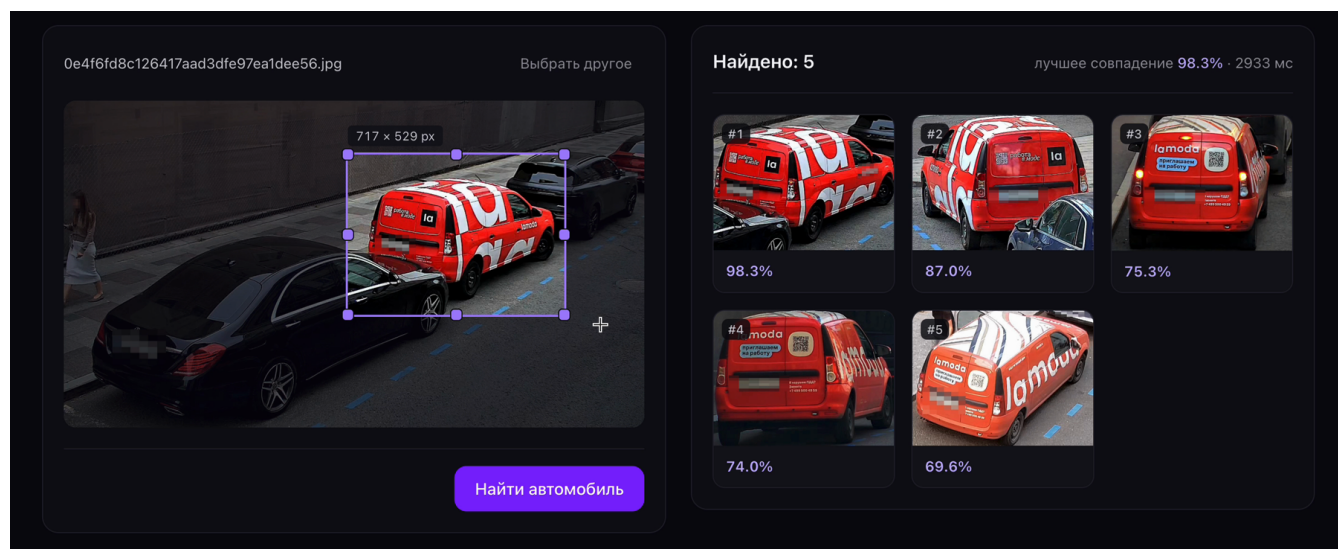


Выделение автомобиля на исходном кадре: координаты bbox сохраняются в пикселях изображения. Кадр предоставленного скринкаста.

Polling и состояния интерфейса

После POST клиент опрашивает GET с задержкой 400 мс, далее экспоненциально увеличивает интервал до 2 с. Общий UI-таймаут - 60 с. Ранний 404 в течение 2 с терпится как возможная гонка между процессами, затем показывается истечение. После timeout интерфейс проверяет /health, чтобы отличить недоступный API от ситуации, когда API жив, а задачу не берёт worker.

Состояние	Что видит пользователь
idle / uploading / polling	Пустая колонка результатов, отправка, ожидание с task_id и таймером.
found	Ранжированные карточки с изображениями, score и временем.
rejected	Нейтральное сообщение «Такого автомобиля нет в галерее».
failed / expired / timedOut	Отдельные сообщения об ошибке обработки, истёкшем ID или ожидании.



Пример найденных кандидатов в интерфейсе. Показанные проценты и 2933 мс относятся только к этому записанному запросу, не к benchmark. Кадр предоставленного скринкаста.

Nginx проксирует /api/ и /health на backend с одного origin, поэтому в Compose не нужен отдельный CORS-настрой на браузере. VITE_DEMO_MODE=true переключает фронтенд на fixtures и вшивается во время build; это демонстрация интерфейса без настоящего поиска, её нельзя выдавать за результат модели.

09. Хранилища и срок жизни состояния

Хранилище	Ключ / сущность	Назначение
Redis	falcon:tasks, falcon:gallery	Две независимые очереди: поиск и индексация.
Redis	task:{id}, result:{id}	Статус и JSON-результат, TTL 1 час.
Redis	gallery:state, gallery:failed	Ожидаемая версия/размер индекса и пропущенные изображения.
Qdrant	gallery	L2-нормированные признаки и payload галереи.
Postgres	gallery_images	Метаданные, в том числе хеш манифеста.
Postgres	search_queries, search_candidates	История запросов и найденных кандидатов.
RustFS	queries/, gallery/	Исходные файлы; в API отдаются presigned URL.

Идентификатор точки Qdrant - UUID5 от `image_id`; исходный ID остаётся в `payload`. Обработка поиска идёт через Redis независимо от истории Postgres: запись истории в `backend` помечена `best-effort`. Параметры `volumes` в `Compose` привязывают сервисное состояние к `data/redis`, `data/postgres`, `data/qdrant`, `data/rustfs`.

```
docker compose down
# Полный локальный сброс служебного состояния:
rm -rf data/postgres data/qdrant data/redis data/rustfs
# data/gallery и data/weights оставьте на месте
```

Сброс уничтожает индекс, очередь, историю и объекты RustFS. При следующем старте `init` снова загрузит галерею и воркеры пересчитают признаки; это не эквивалент простой остановки `docker compose down`.

10. Конфигурация и развёртывание

Compose передаёт параметры через переменные среды. Источник полного перечня и дефолтов - `.env.example`; код валидирует настройки отдельных сервисов при запуске. Таблица показывает ключевые переключатели, а не все внутренние таймауты и реквизиты.

Параметр	По умолчанию	Влияние
GALLERY_DIR	./data/gallery	Монтируемая статическая галерея.
INIT_MODEL_URL, SHA256	HF EVA02	Источник и контроль веса модели.
INIT_CATBOOST_URL, SHA256	HF CatBoost	Источник и контроль refusal head.
INIT_FORCE	false	Полная переиндексация галереи.
INFERENCE_DEVICE	auto	CUDA при доступности, иначе CPU.
INFERENCE_REPLICAS	1	Число независимых worker-процессов.
INFERENCE_COMPILE	false	Опциональный <code>torch.compile</code> на CUDA.
INFERENCE_REFUSAL_CONFIG	eva02_ensemble.yaml	Логика open-set решения.
VITE_DEMO_MODE	false	Frontend fixtures вместо API; нужен rebuild.
FRONTEND_PORT	8080	Внешний порт веб-интерфейса.

Инфраструктурные образы и доступ

Компонент	Образ в Compose	Порт
Redis	redis:7.4.11-alpine	6379
Postgres	postgres:17.6-alpine	5432
Qdrant	qdrant/qdrant:v1.19.1	6333/6334
RustFS	rustfs/rustfs:1.0.0-rc.6	9000/9001
Backend / Frontend	Локальные Dockerfile	8000 / 8080

В Compose порты Redis, Postgres, Qdrant, RustFS и backend опубликованы на хосте. Дефолтные пароли предназначены для локального демо. Перед размещением в общей сети ограничьте публикацию внутренних портов, смените секреты, настройте внешний адрес `S3_PUBLIC_ENDPOINT_URL` и TLS. Это настройка окружения; репозиторий не содержит готового production-профиля с НА.

11. Масштабирование и эксплуатационные границы

API stateless по вычислению модели: тяжёлый forward остаётся в worker. INFERENCE_REPLICAS=N добавляет N участников одной Redis consumer group. Это увеличивает параллелизм задач, но каждая реплика держит отдельную EVA02 и CatBoost, а один worker обрабатывает одну картинку за раз. На CPU стоит ограничить INFERENCE_TORCH_THREADS, иначе реплики конкурируют за ядра; на GPU нужно учитывать VRAM на каждую копию.

Реализовано сейчас	Предлагаемый следующий шаг, не реализован
Redis Streams, consumer group, перехват брошенных задач.	Отдельный Triton/gRPC embedder с общей копией модели и dynamic batching.
Раздельные очереди поиска и gallery; поиск имеет приоритет.	Batch-индексация галереи и измерение очереди при высокой нагрузке.
Проверка версии весов в Qdrant payload.	Разнесение хранилищ и отказоустойчивая кластеризация.

Triton в README описан как вариант развития, а не часть работающего Compose. В текущем демонстрационном сценарии одиночные запросы не собирают batch; добавление отдельного сетевого сервиса само по себе не доказывает ускорения. Для заявлений о производительности нужны замеры end-to-end при указанной нагрузке и железе.

Наблюдаемость

Worker логирует task_id, стадию, статус, отказ, число кандидатов и latency_ms; init и gallery-ready - объём очереди, долю завершения, скорость и ETA. Healthcheck в образе worker проверяет обновляемый heartbeat-файл. Проверяйте вместе docker compose ps, логи и реальный POST→GET: любой одиночный green healthcheck не доказывает качество поиска.

12. Проверки и воспроизводимость

12.1. Минимальная сквозная проверка

- В `docker compose ps` дождаться успешного `init` и `gallery-ready`, работающих `backend`, `frontend` и `inference`.
- Открыть `http://localhost:8080`, загрузить кадр, нарисовать `bbox`, проверить отображение результата или штатного отказа.
- Повторить тот же запрос через `curl`: `POST` обязан дать `202/task_id`, `GET` сначала может дать `202`, затем `200` с `status=done` либо `status=failed`.
- Сверить, что Qdrant collection `gallery` содержит точки с текущим `model_version`; при ошибках `gallery` смотреть `gallery:failed` и `worker logs`.
- Проверить, что `score` упорядочен по убыванию, `rank` начинается с 1, а при `rejected=true` кандидатов нет.

12.2. Тесты и сборка разработчика

```
git submodule update --init
uv sync
uv run pytest
uv run ruff check .
cd frontend
npm ci
npm run test
npm run lint
npm run build
```

`pytest` в корневом `pyproject.toml` охватывает `backend/tests`, `inference/tests`, `init/tests` и `common/tests`; интеграционные проверки с реальным `Postgres` помечены `integration` и требуют `POSTGRES_TEST_DSN`. Frontend использует `Vitest`, `ESLint` и `TypeScript build`. Этот документ фиксирует команды, но не подменяет протокол отдельного CI-прогона.

Для стабильного сравнения алгоритма используйте ML-руководство: там описаны `5-fold identity-disjoint validation` и метрики `refusal`. Скринкаст интерфейса, единичный `score` и время одного запроса не являются оценкой качества или пропускной способности сервиса.

13. Диагностика типовых ошибок

Симптом	Проверка / действие
init не стартует: нет галереи	Проверьте GALLERY_DIR, manifest.csv, images/ и docker compose logs init.
Весы не скачиваются	Проверьте URL/доступ к HF, INIT_HF_TOKEN при необходимости, SHA256 и data/weights.
gallery-ready долго ждёт	Смотрите docker compose logs -f inference gallery-ready; на CPU одна картинка может считаться долго. Проверьте версию весов и stalled warning.
POST 422	Проверьте multipart поля, исходный формат изображения, лимит 20 МиБ, bbox хуwh и минимум 16 px после clamp.
POST 503	Проверьте RustFS, bucket, ключи доступа и endpoint.
GET остаётся 202	Проверьте, что worker запущен, модель загрузилась и читает falcon:tasks.
GET 404	Нет ID либо истёк часовой TTL. Отправьте новый POST.
Frontend 502 после backend restart	Проверьте backend и nginx; конфиг резолвит backend через Docker DNS, затем повторите запрос.
Картинки кандидатов не открываются	Проверьте S3_PUBLIC_ENDPOINT_URL, доступность порта 9000 из браузера и TTL подписанных URL.
Смена весов без обновления индекса	Проверьте SHA256 модели в init/worker и model_version; повторите bootstrap, при необходимости INIT_FORCE=true.

Для сбора контекста достаточно `docker compose ps`, `docker compose logs --tail=200 init gallery-ready inference backend frontend` и одного воспроизводимого `task_id`. Не публикуйте в отчётах секреты из `.env` и приватные исходные кадры.

14. Карта исходников

Тема документа	Проверяемые файлы
Архитектура и запуск	README.md, docs/images/architecture.jpg, docker-compose.yaml, docker-compose.gpu.yaml, .env.example.
HTTP и валидация	backend/app/routers/search.py, backend/src/services/search.py, backend/src/services/validation.py.
Очереди и договоры	common/src/configs/constants.py, common/src/configs/schemas.py, common/src/redis/client.py.
Модель и отказ	inference/src/models/reid.py, factory.py, refusal_head.py, processor.py, pipeline/.
Индексация	init/src/pipeline.py, gallery.py, steps/, wait.py, inference/src/gallery.py.
UI и nginx	frontend/src/components/, lib/useSearchTask.ts, api/search.ts, deploy/nginx.conf.
Тесты	backend/tests, inference/tests, init/tests, common/tests, frontend/tests.